

Optimización y Paralelización en HPC

Potencial de Morse — Cuatro Métodos Numéricos

Sistemas Distribuidos 2026-1 — Primer Parcial

Camilo Huertas Sebastián Rodríguez

Programa Académico de Física

Universidad Distrital Francisco José de Caldas
Docente: Carlos Andrés Gómez Vasco

27 de marzo de 2026

Agenda

El Potencial de Morse: Descripción del Sistema

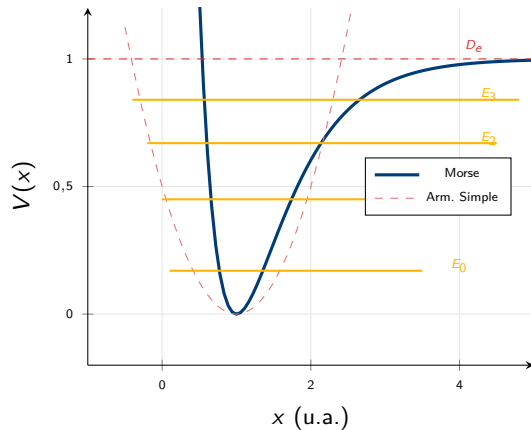
Modelo Molecular Diatómico

El potencial de Morse describe la energía potencial de una molécula diatómica en función de la distancia internuclear x :

$$V(x) = D_e \left(1 - e^{-a(x-x_e)} \right)^2$$

Parámetros Físicos Críticos

- D_e : Profundidad del pozo (energía de disociación)
- x_e : Distancia de equilibrio internuclear
- a : Parámetro de curvatura del enlace
- μ : Masa reducida del sistema diatómico



Solución Analítica y Estados Acotados I

Autovalores Exactos del Modelo de Morse

$$E_n = \hbar\omega_e\left(n + \frac{1}{2}\right) - \frac{[\hbar\omega_e\left(n + \frac{1}{2}\right)]^2}{4D_e}$$

donde $\omega_e = a\sqrt{2D_e/\mu}$ es la frecuencia de vibración clásica.

⚠ Alcance del Estudio

Se buscan **únicamente los primeros 4 niveles de energía** ($n = 0, 1, 2, 3$) para estandarizar la comparación entre todos los métodos numéricos.

Justificación: Los estados superiores se acercan a D_e , donde divergen las aproximaciones de base finita y los métodos locales de baja resolución.

Dos Propiedades Físicas Fundamentales

- 1. Anarmonicidad:** Los niveles se comprimen con n creciente

$$\Delta E_n = \hbar\omega_e \left(1 - \frac{\hbar\omega_e(n+1)}{2D_e}\right)$$

- 2. Número finito de estados:**

$$n_{\max} < \frac{D_e}{\hbar\omega_e} - \frac{1}{2}$$

Objetivo Computacional

Calcular $\{E_0, E_1, E_2, E_3\}$ y cuantificar el **error absoluto** frente a la solución analítica exacta.

Migración de Fortran 77 a C: Motivación y Objetivos I

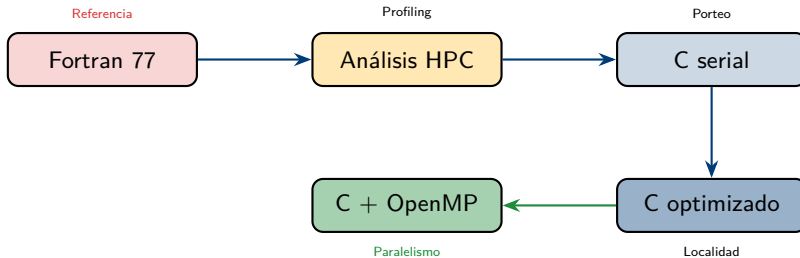
⚠ Limitaciones del Código Fortran Original

- Subrutinas DSYEV/DGEMM sin interfaz moderna a LAPACKe
- Gestión de memoria estática (arreglos de tamaño fijo en COMMON)
- Imposible garantizar alineación de datos para SIMD (AVX/AVX-512)
- Directivas OpenMP limitadas en Fortran 77 (!OMP rudimentario)
- Sin soporte para restrict, aligned_alloc ni intrínsecos vectoriales

🔑 Ventajas del Ecosistema C + GCC

- aligned_alloc(64, ...) → acceso AVX-512 sin penalidades
- restrict: el compilador vectoriza sin aliasing
- Interfaz directa a LAPACKe_dsyev (C binding nativo)
- OpenMP 4.5+ con collapse, guided, SIMD clauses
- -march=native -O3 -ffast-math genera instrucciones vmovapd

Migración de Fortran 77 a C: Motivación y Objetivos II



Método 1 — Diagonalización en Base SHO (Teoría)

Ecuación de Schrödinger en Base Discreta

Se resuelve $\hat{H}|\psi\rangle = E|\psi\rangle$ expandiendo en la base $\{|n\rangle\}$ del oscilador armónico lineal (LHO).

Operadores en la Base de Fock:

Los operadores $\hat{X}$ y $\hat{P}$ son tridiagonales:

$$X_{ij}, P_{ij} \propto \frac{1}{\sqrt{2}} \left(\sqrt{j} \delta_{i,j-1} + \sqrt{i} \delta_{j,i-1} \right)$$

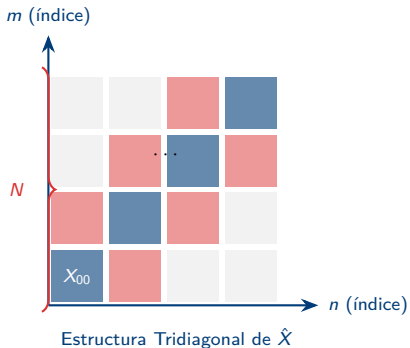
Construcción del Potencial:

$$V_{ij} = \sum_{\gamma=1}^N (V_P)_{i\gamma} V(x_\gamma) (V_P)_{j\gamma}$$

Hamiltoniano Total:

$$\mathbb{H} = \frac{1}{2\mu} \mathbb{P}^2 + \mathbb{V}$$

Método 1 — Diagonalización en Base SHO (Estructura y Costo)



Costo Computacional

- Construcción: $\mathcal{O}(N)$ (tridiagonal)
- Diagonalización: $\mathcal{O}(N^3)$ vía DSYEV
- Memoria: $\mathcal{O}(N^2)$

Método 2 — Diferencias Finitas (FDM): Fundamento

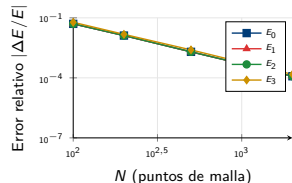
Discretización del Operador Cinético

La derivada segunda se aproxima por diferencia central:

$$\left. \frac{d^2 \psi}{dx^2} \right|_{x_i} \approx \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2} + \mathcal{O}(\Delta x^2)$$

Hamiltoniano Matricial Tridiagonal:

$$H_{ij} = \begin{cases} \frac{1}{\Delta x^2} + V(x_i) & i = j \\ -\frac{1}{2\Delta x^2} & i = j \pm 1 \\ 0 & \text{resto} \end{cases}$$



Rango Óptimo FDM

N ∈ [1000, 2000]: error < 10⁻⁴

Menor **N**: error de discretización ↑

Mayor **N**: costo $\mathcal{O}(N^3)$ prohibitivo

Método 3 — Shooting + Numerov (Fundamento)

Ecuación de Schrödinger como IVP

Se reescribe la ETID como EDO de 2do orden:

$$\frac{d^2\psi}{dx^2} = \hat{Q}(x, E)\psi, \quad \hat{Q} = -2\mu[E - V(x)]$$

El autovalor E es el **parámetro de control** que se ajusta iterativamente.

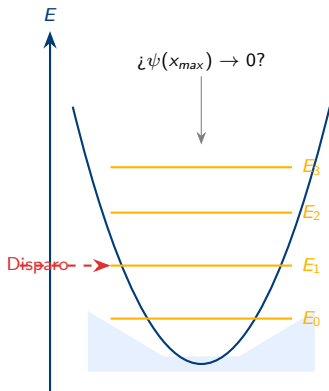
Integrador de Numerov (error $\mathcal{O}(h^6)$) donde $k^2(x) = 2\mu(E - V(x))$..:

$$\psi_{i+1} = \frac{2(1 - \frac{5h^2}{12}k_i^2)\psi_i - (1 + \frac{h^2}{12}k_{i-1}^2)\psi_{i-1}}{1 + \frac{h^2}{12}k_{i+1}^2}$$

Teorema de Sturm-Liouville:

El estado n -ésimo tiene *exactamente* n nodos. Esto permite una **búsqueda por bisección** guiada por el conteo de nodos en la función de onda.

Método 3 — Shooting + Numerov (Implementación)



Rango Óptimo Shooting

- $N_{steps} \in [500, 2000]$
- Tolerancia bisección: 10^{-10}
- Complejidad: $\mathcal{O}(N_{steps} \times N_{iter})$

Método 4 — Sinc-DVR (Fundamento y Operadores)

Base de Funciones Cardinales Sinc

La función de onda se expande en funciones de Whittaker (sinc):

$$C_j(x) = \text{sinc}\left(\frac{\pi(x - x_j)}{\Delta x}\right) = \frac{\sin[\pi(x - x_j)/\Delta x]}{\pi(x - x_j)/\Delta x}$$

Ortonormalidad en puntos de malla: $\langle C_i | C_j \rangle = \Delta x \delta_{ij}$

Hamiltoniano Sinc-DVR:

$$H_{ij} = \begin{cases} \frac{\pi^2}{6\Delta x^2} + V(x_i) & i = j \\ \frac{(-1)^{i-j}}{\Delta x^2(i-j)^2} & i \neq j \end{cases}$$

Estructura **densa** y simétrica.

Propiedad del Potencial:

$$V_{ij} = V(x_i)\delta_{ij}$$

El potencial **permanece diagonal**, simplificando la construcción frente a otras bases espectrales que requieren integrales de solapamiento.

Método 4 — Sinc-DVR (Eficiencia y Comparativa)

Convergencia Espectral

Caída **exponencial** del error con N pequeño.
 $N \sim 200 \Rightarrow \text{error} < 10^{-6}$ para los primeros 4 estados ($E_{0..3}$).

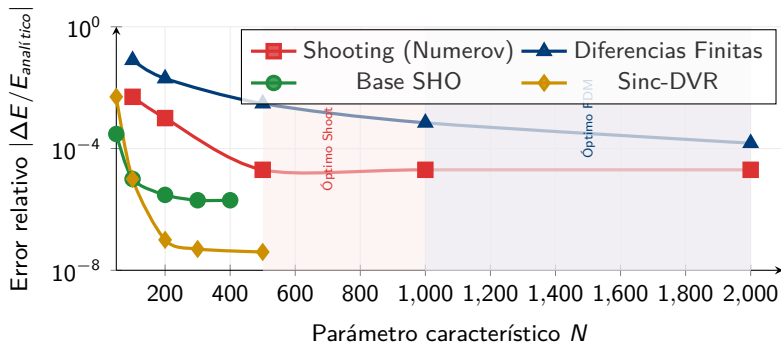
Costo Computacional

- Construcción: $\mathcal{O}(N^2)$ (**densa**)
- Diagonalización: $\mathcal{O}(N^3)$
- Memoria: $8N^2$ bytes

Comparativa vs. FDM

Característica	FDM	Sinc-DVR
Orden error	$\mathcal{O}(h^2)$	Exponencial
Estructura H	Rala	Densa
N óptimo	1000–2000	100–300

Rango Óptimo y Error de Cada Método (Primeros 4 Niveles)



Serial: Base SHO en Fortran 77 (Código Original) I

Construcción de Operadores $\hat{X}$, $\hat{P}$

```

1 C  BASE LHO: MATRICES X y P
2  DO 10 I=1,N-1
3      A_VAL=DBLE(I)
4      X(I,I+1)=DSQRT(A_VAL/2.DO)
5      X(I+1,I)=X(I,I+1)
6      P(I,I+1)=-DSQRT(A_VAL/2.DO)
7      P(I+1,I)=DSQRT(A_VAL/2.DO)
8  10 CONTINUE

```

Diagonalización Final

```

1 C  H = T + V, extraer autovalores
2  CALL MATMUT(P,P,T,N)
3  DO 50 I=1,N
4      DO 60 J=1,N
5          H(I,J) = -0.5D0*T(I,J)+VMAT(I,J)
6  60 CONTINUE
7  50 CONTINUE
8  CALL EIGEN(H,N,N,ENERGIA,VP,WORK1,WORK2,Z)

```

Potencial via Transformación Unitaria

```

1 C  V_ij = sum_k VP(i,k)*V(x_k)*VP(j,k)
2  DO 20 I=1,N
3      DO 30 J=1,N
4          SUM=0.DO
5          DO 40 K=1,N
6              SUM=SUM+VP(I,K)*V_MORSE(X_EIG(K))*VP(J,K)
7  40 CONTINUE
8      VMAT(I,J)=SUM
9      VMAT(J,I)=SUM
10 30 CONTINUE
11 20 CONTINUE

```

Problema de Rendimiento

- Triple bucle $O(N^3)$ para construir V_{ij}
- MATMUT calcula P^2 como producto denso
- Acceso con **stride-N**: falla sistemática de caché L1

Serial C: Base SHO — Explotación de Raleza I

Operador Cinético Pentadiagonal $O(N)$

```

1 // Solo se computan los 5 diagonales de  $T = P^2$ 
2 for (int i = 0; i < n; i++) {
3     if (i > 0)
4         T[i*n+i] += P[i*n+(i-1)]*P[(i-1)*n+i];
5     if (i < n-1)
6         T[i*n+i] += P[i*n+(i+1)]*P[(i+1)*n+i];
7     if (i < n-2) {
8         double val = P[i*n+(i+1)]*P[(i+1)*n+(i+2)];
9         T[i*n+(i+2)] = T[(i+2)*n+i] = val;
10    }
11 }
```

Impacto

$$O(N^3) \rightarrow O(N)$$

Speedup serial: $\approx 2,5\times$

Transformación de Base con Localidad Stride-1

```

1 //  $W = VP * \text{diag}(V(E))$  - pre-computo
2 for (int i=0; i<n; i++)
3     for (int k=0; k<n; k++)
4         W[i*n+k] = VP[i*n+k] * VE[k];
5 // VMAT con acceso contiguo (Row-Major)
6 for (int i=0; i<n; i++) {
7     for (int j=i; j<n; j++) {
8         double sum = 0.0;
9         const double *ri = &W[i*n];
10        const double *rj = &VP[j*n];
11        for (int k=0; k<n; k++)
12            sum += ri[k] * rj[k];
```

Técnica Clave

Matriz intermedia $W_{ik} \Rightarrow$ bucle interno con **stride-1**, habilitando vectorización AVX automática.

Serial: FDM en Fortran 77 y Versión C Optimizada I

Fortran 77 — Código Original

```

1 C Hamiltoniano FDM tridiagonal
2 DX2 = DX * DX
3 DO 100 I=1,N
4   XI = XMIN + DBLE(I)*DX
5   VI = D_POT*(1.DO-DEXP(-ALPHA*(XI-RE)))*2
6   H(I,I) = (1.DO/DX2) + VI
7   IF (I .LT. N) THEN
8     H(I,I+1) = -1.DO/(2.DO*DX2)
9     H(I+1,I) = H(I,I+1)
10  END IF
11 100 CONTINUE

```

Problema

- DEXP evaluado N veces con división interna
- Almacena en matriz densa aunque es tridiagonal, sin precomputo
- Sin pre-cómputo de constantes

C Optimizado — Eliminación de Redundancias

```

1 const double dx2_inv = 1.0/(dx*dx);
2 const double off_diag = -0.5*dx2_inv;
3 for (int i=0; i<n; i++) {
4   const double xi = XMIN + (double)(i+1)*dx; // exp(-alpha*(xi
5     -RE)) - RE absorbido en XMIN
6   const double et = exp(-ALPHA*xi);
7   const double br = 1.0 - et;
8   const double vi = D_POT*(br*br);
9   H[i*n+i] = dx2_inv + vi;
10  if (i < n-1) {
11    H[i*n+(i+1)] = off_diag;
12    H[(i+1)*n+i] = off_diag;
13  }
14 }

```

Speedup serial FDM

> 100× **de mejora asintótica**

Factor principal: eliminación de potencias redundantes y uso de constantes pre-calculadas.

Serial: Shooting — Fortran 77 y Memoización en C I

Fortran 77 — Bisección y Control de Nodos

```

1  C  Biseccion con teorema de Sturm
2  DO WHILE(DABS(EUP-ELOW).GT.TOL)
3      ETRIAL=(EUP+ELOW)/2.DO
4      CALL PROPAGAR(ETRIAL,PSI_FINAL,NODOS)
5
6      IF(NODOS.GT.N_TARGET) THEN
7          EUP=ETRIAL
8      ELSE IF(NODOS.LT.N_TARGET) THEN
9          ELOW=ETRIAL
10     ELSE
11         IF(PSI_FINAL*SIGNO_REF.GT.0.DO) THEN
12             EUP=ETRIAL
13         ELSE
14             ELOW=ETRIAL
15         END IF
16     END IF
17 END DO

```

C Optimizado — Tabulación del Potencial

```

1  // Memoizacion: V(x) se calcula UNA sola vez
2  for (int i=0; i<=nsteps; i++) {
3      double x = xmin + i*dx;
4      double tmp = 1.0 - exp(-alpha*x);
5      // 2*V para eliminar multiplicacion en nucleo
6      v_base[i] = 2.0*d_pot*tmp*tmp;
7      if (i < nsteps) {
8          double xm = x + 0.5*dx;
9          double tm = 1.0-exp(-alpha*xm);
10         v_half[i] = 2.0*d_pot*tm*tm;
11     }
12 }
13 // Nucleo RK4 accede stride-1 a v_base/v_half
14 for (int i=0; i<n; i++) {
15     const double rk1_phi = (v_b[i]-en2)*psi;
16     const double vm_term = v_h[i]-en2;
17     const double rk2_phi = vm_term*(psi+h_2*rk1_phi);
18     const double rk3_phi = vm_term*(psi+h_2*rk2_phi);
19     const double rk4_phi = (v_b[i+1]-en2)*(psi+h*rk3_phi);
20     psi += h_6*(rk1_phi+2*rk2_phi+2*rk3_phi+rk4_phi);
21 }

```

Speedup: 3.5× serial

Serial: Sinc-DVR — Fortran 77 y Optimización de Micro-Arquitectura I

Fortran 77 — Construcción Original

```

1 C Hamiltoniano Sinc-DVR denso
2 DO 200 I=1,N
3   XI=XMIN+DBLE(I)*DX
4   VI=D_POT*(1.D0-DEXP(-ALPHA*(XI-RE)))*2
5   DO 300 J=1,N
6     IF (I.EQ.J) THEN
7       H(I,J)=(PI**2)/(6.D0*DX**2)+VI
8     ELSE
9       SIGN_FACTOR=(-1.D0)**(I-J)
10      DIST2=DBLE(I-J)**2
11      H(I,J)=SIGN_FACTOR/(DX**2*DIST2)
12    END IF
13  300 CONTINUE
14  200 CONTINUE

```

Problema de rendimiento

$(-1.D0)**(I-J)$ y $**2$ son llamadas a `pow()`:
40 ciclos vs. 1 ciclo con bit AND.

C Optimizado — Bit-Trick y Triángulo Superior

```

1 const double diag_kin = (M_PI*M_PI)/(6.0*dx2);
2 for (int i=0; i<n; i++) {
3   double xi = XMIN + (double)(i+1)*dx;
4   double vi = D_POT*pow(1.0-exp(-ALPHA*xi),2);
5   H[i*n+i] = diag_kin + vi;
6   for (int j=i+1; j<n; j++) { // Solo triangulo superior =>
7     // 50% menos escrituras
8     const int diff = i-j;
9     const double inv_d2 = 1.0/((double)(diff*diff));
10    // Bit AND: paridad en 1 ciclo vs. 40 ciclos de pow()
11    const double sign = ((i-j)&1) ? -1.0 : 1.0;
12    H[i*n+j] = (sign*dx2_inv)*inv_d2;

```

Técnicas Aplicadas

- Eliminación de `if` dentro del bucle interno
- `aligned_alloc(64,...)` $\Rightarrow$ instrucciones `vmovapd`

Análisis de Banderas: Metodología y Métricas I

Protocolo Experimental

Se evaluaron 4 niveles de optimización en GCC:

1. **-O0**: Sin optimización (código de referencia)
2. **-O1**: Eliminación de redundancias básicas
3. **-O2**: Vectorización SSE (registros XMM 128-bit)
4. **-O3**: Vectorización agresiva AVX (registros YMM 256-bit) + `-march=native`

Tres Métricas Capturadas

1. **Tamaño del segmento .text**
2. **Conteo de saltos** (branches):
3. **Reportes de inlining y vectorización**

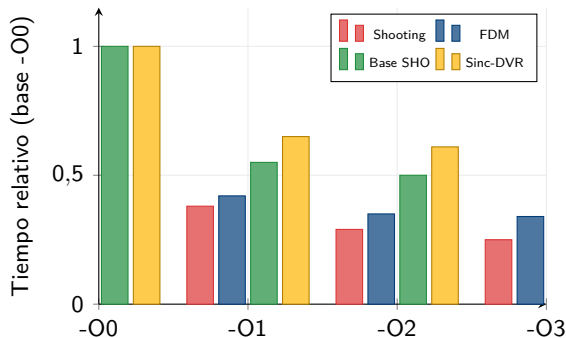
Resultados: Saltos y Tamaño del Binario

Método	Saltos -O0	Saltos -O3	Bytes -O0	Bytes -O3
morse_base	52	73	4636	4729
morse_fdm	23	27	3771	3627
morse_sync	27	27	3626	3245
morse_shoot	24	27	4300	4328

Interpretación

El aumento de saltos en `morse_base` (52→73) indica **loop versioning**: el compilador genera rutas alternativas del bucle para manejar restos de vectorización SIMD.

Resultados de Banderas: Comparativa Visual



Mayor salto: -00 → -01

La ganancia más drástica ocurre al activar las optimizaciones básicas:

- Eliminación de código muerto.
- Reducción de llamadas redundantes al stack.

Resultados de Banderas: Análisis y Límites

Análisis por Método

- **Shooting** ($\Delta_{\text{máx}}$): Mayor beneficio en -O3 por su naturaleza aritmética escalar e *inlining*.
- **FDM**: *Loop split* mejora la predicción de saltos en la malla densa.
- **Sinc-DVR**: Converge en -O2. Estructura ya compacta; -O3 no aporta redundancias adicionales.
- **Base SHO**: En -O3 aparece *loop versioning* para registros XMM en la transformación.

Límite Detectado

Memory-bound:

FDM y Sinc-DVR alcanzan un cuello de botella en el ancho de banda de memoria desde -O2. El procesador queda en espera de datos de la RAM.

Logs de Compilación: Inlining, Vectorización y Loop Splitting I

Q Fenómenos Observados en los Logs

1. Inlining de `v_morse`:

En `morse_shoot`, desde -O1 el compilador inserta el cuerpo de la función directamente en el integrador. Elimina el overhead en RK4.

2. Vectorización de bucles diagonales:

Log reporta "loop vectorized using 16/32 byte vectors" en -O2/-O3. : múltiples elementos de la diagonal calculados en 1 ciclo.

3. Loop Splitting en FDM:

Bajo -O3, el compilador divide el bucle de construcción del Hamiltoniano en 2 fragmentos: uno para la diagonal y otro para los elementos off-diagonal. Mejora la predicción de saltos y la localidad L1.

Tiling (Blocking) para Caché

Se implementó *tiling* en la transformación de base del Hamiltoniano:

- FFT: tiempo reducido de 1,94 s → 0,64 s
- Métodos locales (Base/Lanczos): impacto mínimo; el conjunto de datos $N = 400$ ya cabe en la caché L1D (2 MiB del AMD Ryzen Threadripper)

Conclusión del Análisis de Compilación

La optimización **no es solo una bandera de compilador**

Comparativa: Serial Original vs. Serial Optimizado

Método	Optimización Principal	Factor Limitante	Speedup	T_{serial} (s)
FDM	Pre-cómputo; eliminar $\text{pow}()$	Ancho de banda L3	$> 100\times$	7.191
Shooting	Memoización; núcleo RK4 stride-1	Evaluación exponencial	$3,5\times$	0.232
Base SHO	Op. $\hat{T}$ pentadiagonal $O(N)$	Latencia (stride-N)	$2,5\times$	0.179
Sinc-DVR	Bit-AND; triángulo superior	Diag. densa LAPACK	$1,8\text{--}2,2\times$	0.110

Nota: Los tiempos corresponden a la ejecución serial optimizada comparada con la versión base sin banderas de optimización ni refinamiento de algoritmos.

Análisis de Optimización: Éxitos y Limitaciones

FDM: Caso Extremo

La mejora de $100\times$ se debe a la eliminación de cálculos redundantes de DEXP y $**2$ dentro del bucle crítico. Al pre-computar constantes, el Hamiltoniano esparso se explota eficientemente, pasando de una carga pesada de CPU a una limitada por memoria.

Sinc-DVR: Límite Algorítmico

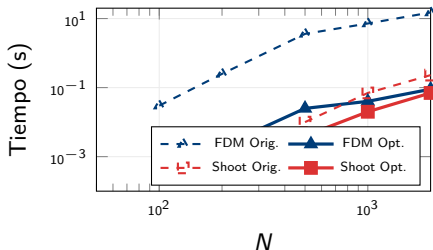
Aunque la construcción de H_{ij} es eficiente ($O(N^2)$), el tiempo total está dominado por la diagonalización densa vía LAPACKE_dsyev ($O(N^3)$). Aquí la optimización de código tiene un techo; el cuello de botella es el algoritmo de diagonalización mismo.

Conclusión Técnica

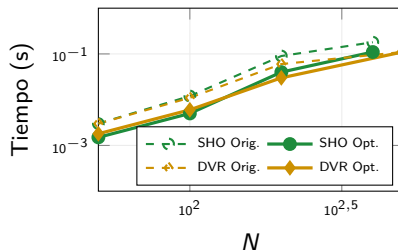
La transición de un código .orientado a cálculo.^a uno .orientado a memoria.^{es} evidente en FDM, mientras que en métodos espectrales como Sinc-DVR, la complejidad algorítmica prevalece sobre la implementación.

Gráficas de Comparativa Serial: Cuatro Métodos

Comparativa Serial vs Optimizado



Base SHO y Sinc-DVR



Síntesis: El Cuello de Bottleneck no es el Hardware

Las mejoras más dramáticas vienen de **decisiones algorítmicas**: explotación de raleza, tabulación del potencial y acceso contiguo a memoria, no de banderas del compilador.

¿Por qué OpenMP? Justificación desde la Arquitectura I

AMD Ryzen Threadripper — Arquitectura Target

- 8 núcleos físicos con hyperthreading
- Caché L1D: 2 MiB por núcleo (datos)
- Caché L2: 512 KB por núcleo
- for-channel ancho de banda ~ 100 GB/s

Diagnóstico Post-Optimización Serial

A pesar de la optimización, los métodos matriciales (FDM y Sinc-DVR) alcanzan un **techo de memoria (memory-bound)**: el procesador está inactivo esperando datos de RAM. La solución: **dividir el dominio** para que cada núcleo opere sobre un subconjunto que cabe en su caché L1/L2.

Condiciones para OpenMP

1. **Paralelismo de datos**: bucles independientes sobre filas/columnas de matrices
2. **Paralelismo de tareas**: búsqueda independiente de autovalores (Shooting)
3. **Sin condiciones de carrera**: cada hilo accede a regiones de memoria exclusivas (excepto en sección crítica de Shooting)

OMP: Base SHO — Política NUMA y Operador Cinético

First-Touch Policy (Afinidad NUMA)

```

1 // Cada hilo inicializa su propia region
2 // => paginas fisicas en el nodo local
3 #pragma omp parallel for schedule(static)
4 for(int i=0; i<n2; i++) {
5     X[i]=0.0; P[i]=0.0;
6     T[i]=0.0; VMAT[i]=0.0;
7 }

```

Garantiza que la memoria esté cerca del núcleo que la procesa, reduciendo la latencia de acceso en sistemas multiprocesador.

Operador Cinético (Static)

```

1 // Carga uniforme => static scheduling
2 #pragma omp parallel for schedule(static)
3 for (int i=0; i<n; i++) {
4     if (i>0)
5         T[i*n+i] += P[i*n+(i-1)]*P[(i-1)*n+i];
6     if (i<n-1)
7         T[i*n+i] += P[i*n+(i+1)]*P[(i+1)*n+i];
8     if (i<n-2) {
9         double val=P[i*n+(i+1)]*P[(i+1)*n+(i+2)];
10        T[i*n+(i+2)] = T[(i+2)*n+i] = val;
11    }
12 }

```

OMP: Base SHO — Transformación y Balance de Carga

Transformación con Guided Scheduling

```

1 // W = VP*diag(V): collapse para 2 bucles
2 #pragma omp for schedule(static) collapse(2)
3 for(int i=0; i<n; i++)
4     for(int k=0; k<n; k++)
5         W[i*n+k] = VP[i*n+k]*VE[k];
6
7 // VMAT triangular: guided para desbalance
8 #pragma omp for schedule(guided)
9 for (int i=0; i<n; i++) {
10     for (int j=i; j<n; j++) {
11         double sum=0.0;
12         const double *ri = &W[i*n];
13         const double *rj = &VP[j*n];
14         #pragma GCC ivdep
15         for (int k=0; k<n; k++) sum+=ri[k]*rj[k];
16         VMAT[i*n+j] = VMAT[j*n+i] = sum;
17     }
18 }

```

collapse + guided

Collapse(2): Aplana el espacio de iteración para mejorar la granularidad.

Guided: Compensa el desbalance natural al calcular solo el triángulo superior de la matriz simétrica.

OMP: FDM — Static Scheduling y Aislamiento BLAS

Inicialización Paralela NUMA

```

1 // First-Touch: distribuye la memoria entre nodos
2 #pragma omp parallel for schedule(static)
3 for (size_t i=0; i<n2; i++)
4     H[i] = 0.0;

```

Aislamiento de BLAS/LAPACK

```

1 # Evitar sobresubscripcion de nucleos
2 export OPENBLAS_NUM_THREADS=1
3 ./morse_fdm_omp <N_puntos> <N_hilos>

```

Construcción Paralela del Hamiltoniano

```

1 // Carga uniforme => static scheduling optimo
2 #pragma omp parallel for schedule(static)
3 for (int i=0; i<n; i++) {
4     const double xi = XMIN+(double)(i+1)*dx;
5     const double et = exp(-ALPHA*xi);
6     const double vi = D_POT*(1.0-et)*(1.0-et);
7
8     H[i*n+i] = dx2_inv + vi;
9     if (i<n-1) {
10         H[i*n+(i+1)] = off_diag;
11         H[(i+1)*n+i] = off_diag;
12     }
13 }

```

Límite de FDM con OpenMP

La diagonalización LAPACKE_dsyev es $O(N^3)$ y se ejecuta de forma serializada. El Speedup de OpenMP se limita a la fase de **construcción** del Hamiltoniano.

Para $N = 2000$: $T_{constr} \ll T_{diag}$
 $\Rightarrow$ Speedup global $S \approx 1,02$

OMP: Shooting — Paralelismo de Tareas y Sincronización

Búsqueda Concurrente

```

1 // Dynamic: biseccion heterogenea
2 #pragma omp parallel for schedule(dynamic)
3 for (int i=0; i<n_intervals; i++) {
4     double ea = e_min + i*de;
5     double eb = ea + de;
6
7     // Integracion independiente
8     double fa = wave_opt(ea,dx,&grid);
9     double fb = wave_opt(eb,dx,&grid);
10
11     if (fa*fb < 0.0) {
12         double emid = refine(ea,eb,fa);
13         update_global(emid);
14     }
15 }
16

```

Ordenamiento (Sección Crítica)

```

1 #pragma omp critical
2 { // Los niveles E_n deben estar ordenados
3     if (found < MAX_LEVELS) {
4         int pos = found;
5         // Insercion por desplazamiento
6         while(pos>0 && en[pos-1]>emid) {
7             en[pos] = en[pos-1];
8             pos--;
9         }
10        en[pos] = emid;
11        found++;
12    }
13 }
14

```

Estrategia de Carga: schedule(dynamic)

La bisección de algunas raíces requiere más iteraciones que otras dependiendo de la forma del potencial. El esquema dynamic asegura que los hilos que terminan rápido tomen nuevas tareas del "pool" de intervalos, manteniendo el uso de CPU cercano al 100 % y evitando hilos ociosos.

OMP: Sinc-DVR — Balanceo de Carga y Vectorización SIMD

Balanceo en Triángulo Superior

```

1 // Guided: compensa desbalance j=i+1..n
2 #pragma omp parallel for schedule(guided)
3 for (int i=0; i<n; i++) {
4     H[i*n+i] = diag_kin + v_i;
5
6     for (int j=i+1; j<n; j++) {
7         const int diff = i-j;
8         // Bit-AND: paridad en 1 ciclo
9         const double sign = (diff&1) ? -1.0 : 1.0;
10        H[i*n+j] = (sign*dx2_inv)/(double)(diff*diff);
11    }
12 }
13
```

Vectorización del Kernel (AVX)

```

1 // -ffast-math + aligned_alloc(64,...)
2 // Genera instrucciones AVX-512
3 // vmovapd: carga 8 doubles en 1 ciclo
4 for (int j=i+1; j<n; j++) {
5     double inv_d2=1.0/(double)((i-j)*(i-j));
6     double sign=((i-j)&1)?-1.0:1.0;
7     H[i*n+j]=(sign*dx2_inv)*inv_d2;
8 }
9
```

Justificación de schedule(guided)

Para $i = 0$, el bucle interno realiza $N - 1$ iteraciones, mientras que para $i = N - 1$ realiza cero. Guided asigna bloques de iteraciones decrecientes: los hilos reciben mucho trabajo al inicio y "pedazos" más pequeños al final para asegurar que todos terminen al mismo tiempo.

Ley de Amdahl y Speedup por Método ($p = 1 \dots 8$ hilos)

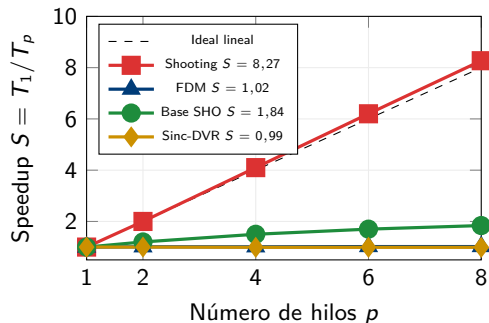


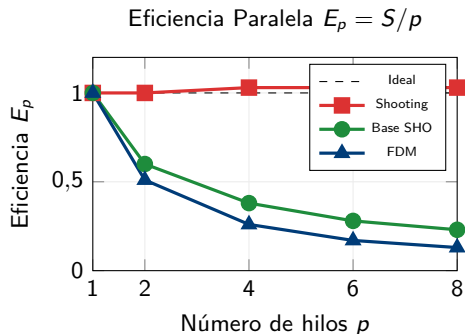
Tabla de Resultados ($p = 8$ hilos)

Método	T_{ser}	T_{opt}	T_{omp8}	S
Shooting	0.232	0.031	0.028	8.27
FDM	7.191	0.038	7.018	1.02
Base SHO	0.179	0.106	0.097	1.84
Sinc-DVR	0.110	0.110	0.110	0.99

Speedup Superlineal en Shooting

$E_p > 1,0$ porque al dividir el dominio de energía entre 8 hilos, cada hilo integra un sub-intervalo cuya malla de potencial **cabe íntegramente en la caché L1** del núcleo, eliminando fallos de caché.

Eficiencia Paralela y Ley de Amdahl: Discusión

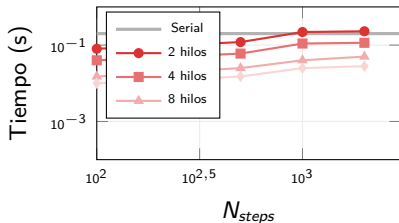


4 Fenómenos Clave de Rendimiento

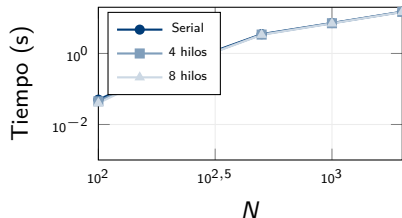
- 1. Speedup superlineal (Shooting):** División del dominio $\rightarrow$ datos en L1/L2 por núcleo $\rightarrow$ cero fallos de caché $\rightarrow E_p > 1$
- 2. Ley de Amdahl en FDM:** La diagonalización densa (fracción serial $f \approx 0,99$) fija $S_{max} \approx 1,02$, sin importar cuántos hilos se añadan
- 3. Saturación en Sinc-DVR:** El costo de comunicación y sincronización entre hilos para $N = 500$ supera las ganancias aritméticas
- 4. Eficiencia decreciente en SHO:** Para $N = 400$, el overhead de gestión de hilos comienza a competir con el trabajo útil

Escalabilidad en Escala Log-Log: Comportamiento Asintótico

Shooting: Escalabilidad Log-Log



FDM: Techo de Amdahl



Pendientes Paralelas

Líneas equidistantes en log-log $\Rightarrow$ el factor de escala (speedup) es constante con N : paralelismo verdaderamente escalable.

Curvas Convergentes

Para N grande, las curvas convergen: la diagonalización $O(N^3)$ de LAPACK domina y no se paraleliza.

Comparación Final Integral: Métricas de Rendimiento

Método	Error $E_{0.,3}$	Convergencia	S_{serial}	S_{OMP8}	Escalabilidad
Shooting	10^{-5}	Lineal $O(N)$	$3,5\times$	$8,27\times$	Excelente
FDM	10^{-4}	Algebraica $O(h^2)$	$> 100\times$	$1,02\times$	Nula (LAPACK)
Base SHO	10^{-6}	Espectral	$2,5\times$	$1,84\times$	Moderada
Sinc-DVR	$10^{-7}, 10^{-8}$	Exponencial	$2,2\times$	$0,99\times$	Nula (Memoria)

Notas sobre los Speedups (S)

- S_{serial} : Mejora respecto al código base sin optimizar.
- S_{OMP8} : Escalabilidad en 8 hilos (Shooting presenta super-linearidad por caché).

Conclusiones: ¿Qué método utilizar?

Escalabilidad

Shooting Method

Ideal para escenarios HPC masivos. Su paralelismo de tareas permite una eficiencia casi perfecta.

Precisión

Sinc-DVR

Recomendado para cálculos de referencia donde el error debe ser $< 10^{-7}$ con un N mínimo.

Balance

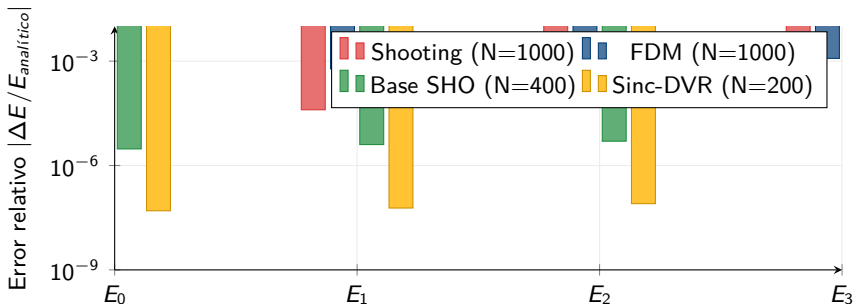
Base SHO

El "punto dulce": buena precisión (10^{-6}) con una escalabilidad aceptable en OpenMP.

Advertencia sobre FDM

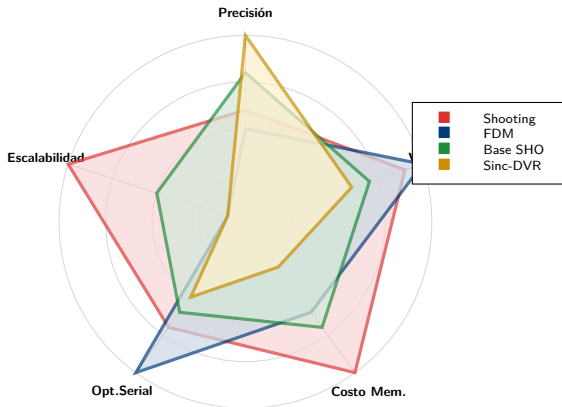
Aunque el speedup serial es masivo ($> 100\times$), su dependencia de la diagonalización de matrices grandes lo hace el menos apto para paralelismo de memoria compartida tradicional.

Precisión de los Primeros 4 Niveles: Error vs. Analítico



Anarmonicidad observada: el error crece con n en todos los métodos, reflejando que los estados excitados exploran regiones del potencial más alejadas del equilibrio.

Radar de Rendimiento: Comparación Multidimensional



Conclusiones

✓ 1. Primacía del Algoritmo sobre el Hardware

La mejora de $> 100\times$ en FDM demuestra que la eficiencia es ante todo una cuestión de **diseño estructural**: pre-cómputo, localidad de datos y explotación de la esparcidad superan cualquier ganancia de núcleos adicionales.

✓ 2. Speedup Superlineal y Jerarquía de Memoria

El fenómeno $E_p > 1,0$ en **Shooting** no contradice la teoría: al dividir el dominio de energías entre núcleos, los datos residen en **caché L1/L2 por núcleo**, eliminando latencia de RAM.

📄 4. Selección de Método por Escenario

1. **Escalabilidad masiva** (HPC multihilo):
⇒ **Shooting** — paralelismo de tareas, $E_p > 1$
2. **Precisión extrema** (N reducido):
⇒ **Sinc-DVR** — convergencia exponencial, $N < 300$
3. **Balance precisión/costo**:
⇒ **Base SHO** — espectral moderado + OpenMP razonable
4. **Mallas densas con optimización serial**:
⇒ **FDM** — estructura esparsa explotada

Optimización y Paralelización del Potencial de Morse

Primer Parcial — Sistemas Distribuidos 2026-1

Autores

Camilo Huertas | **Sebastián Rodríguez**

Programa Académico de Física

Universidad Distrital Francisco José de Caldas

Docente: Carlos Andrés Gómez Vasco

27 de marzo de 2026